

DRL Advertising Optimization System

Architecture Reference — **Version 2.1.0** | `backend/services/ai_optimization/drl/` | 25 Python modules

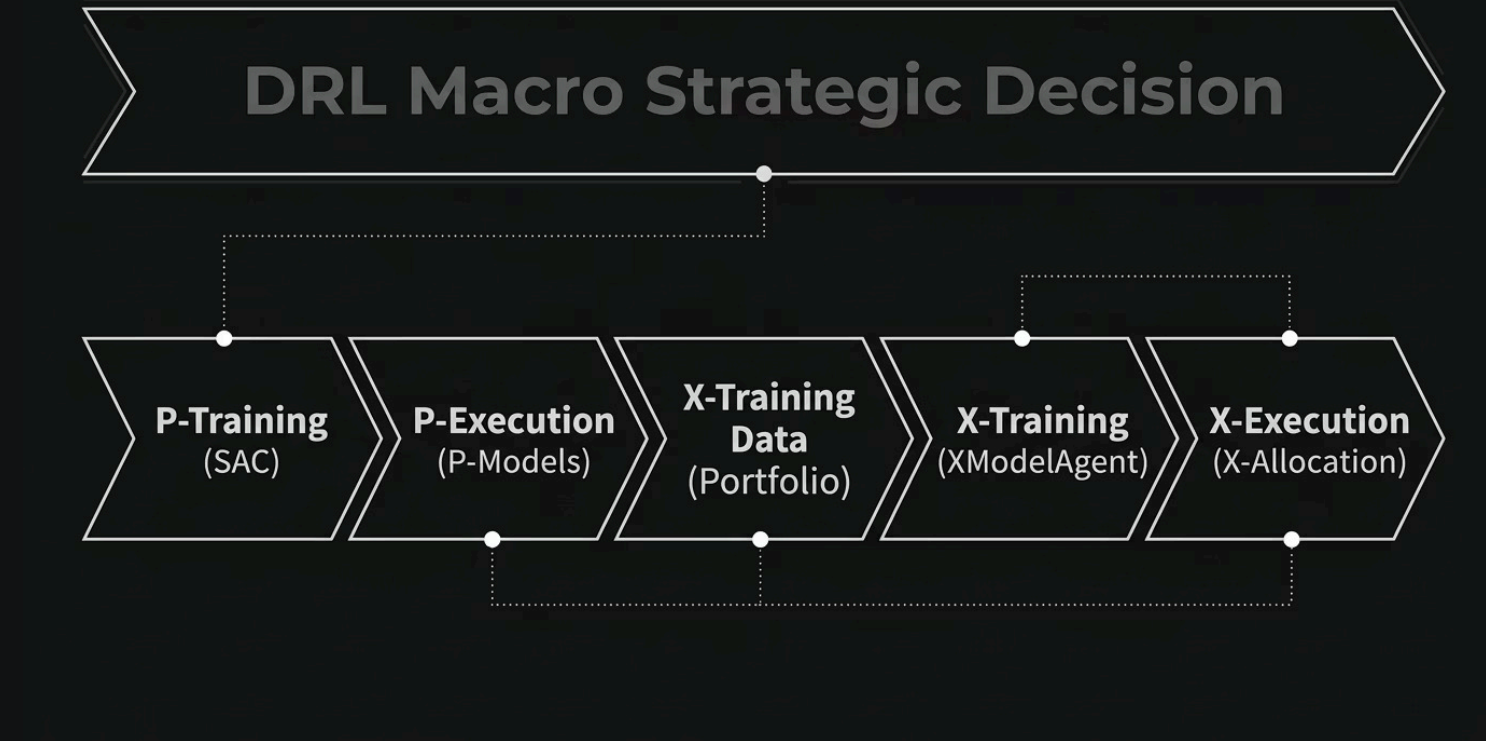
M1-M5 ARCHITECTURE

CROSS-PLATFORM DRL

HYBRID DRL+LLM

System Overview: M1–M5 Architecture

A two-tier DRL architecture — per-platform **P-Models** for campaign-level optimization and a cross-platform **X-Model** for portfolio-level budget allocation — integrated with an LLM layer for tactical execution.



Training Pipeline

M1 → M3 → M4: Per-platform SAC training feeds portfolio-level transitions into X-Model training.

Execution Pipeline

M5 → M2: X-Model selects cross-platform allocation; per-platform P-Models handle campaign-level decisions.

MODULES 1 & 2

Single Platform Model P

Module 1 — Training

Receives **P-training data Results** and **P-Execution data**. Contains a **P-Training DRL** block with Training Parameters. Feeds into the **P-Model** (star icon).

Inputs include History (Multiple P) and Campaign Design parameters: Budget, Cost Per Impression, Audience size, etc.

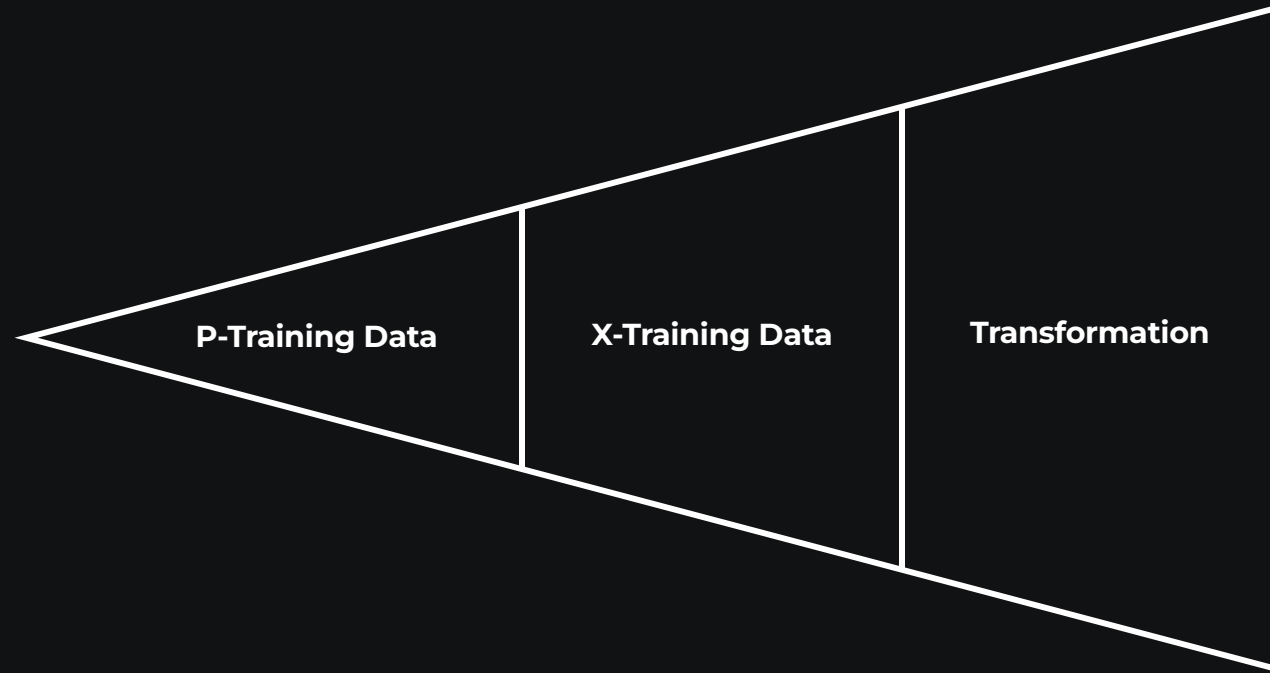
Module 2 — Execution

Contains the **P-Model**, a **P-Execution Data** block, and a **UI** block with P_e (xecution) Parameters. P-Execution Data feeds into **P-Results** and provides feedback to the UI block.

Output: P-Results → P-Campaign Design & Results forecasts.

Creating X Training Data

The P-Training Data may be used for training the X-Model — most likely some transformation is required, involving modeling and data synthesis, or possibly simple aggregations.



P-Training DRL → X-Training DRL

The X-Training process may require calling the P-Training Process — but ideally this should be avoided.

Open Question

The exact transformation method (modeling, synthesis, or aggregation) remains to be determined.

?

MODULES 4 & 5

Cross-Platform Model X

Module 4 — X-Training

X-training data → X-Training DRL → **X-Model!** (star). Inputs also include P-Execution data and Multiple P context. X-Platform Campaign Design parameters: Budget, Constraints, Results.

Module 5 — X-Execution

X-Model → P-Execution Process → **X-Results** (octagon). Xe(xecution) Parameters via UI feed into X Campaign Design & performance forecasts.

File Index: 25 Python Modules

File	Purpose
config.py	Central configuration — 6 dataclasses, 2 enums
state_action.py	State/action definitions — 42-dim state, hybrid action
networks.py	Neural network architectures (Actor, Critic, Value)
sac_agent.py	SAC agent with CQL support
replay_buffer.py	PER with SumTree/MinTree
reward_functions.py	Multi-objective reward — 11 components
offline_trainer.py	Offline CQL training from historical data
safe_agent.py	Production safety wrapper + rollback
hybrid_optimizer.py	DRL+LLM hybrid pipeline
continuous_learning.py	Online/batch learning engine
ab_testing.py	Experiment lifecycle + statistical analysis
cross_platform_optimizer.py	Heuristic budget allocation + portfolio orchestration
cross_platform_drl_engine.py	Top-level DRL orchestrator — 3 strategy modes
x_model.py	Cross-platform DRL agent — 70-dim state, 5-dim action
x_training.py	M4 trainer — behavior cloning + SAC
x_training_data.py	M3 transition builder from portfolio snapshots
xai_narrator.py	Plain-English narrative generation
audience_constraints.py	Audience segment budget allocation + frequency capping
benchmark_model.py	Ridge regression campaign forecaster
forecast_feedback.py	Forecast-vs-actual accuracy tracking + auto-refit
bigquery_loader.py	BigQuery data extraction + normalization
platform_model_registry.py	One SACAgent per platform (M1/M2 infra)
train.py	Mock environment + training loop
train_bigquery_offline.py	End-to-end BigQuery offline training pipeline

Configuration Layer

`config.py` — 362 lines. Six dataclasses and two enums define all system hyperparameters, guardrails, reward weights, and strategy settings.

OptimizationGoal

ROAS, CPA, CONVERSIONS, CTR,
REVENUE, PROFIT

ActionType

BID_ADJUSTMENT,
BUDGET_ADJUSTMENT,
AUDIENCE_EXPANSION,
AUDIENCE_REFINEMENT,
CREATIVE_ROTATION, PLATFORM_SHIFT

AllocationStrategy

DRL_PRIMARY, HEURISTIC_PRIMARY,
DUAL_BENCHMARK

DRLConfig — Core Hyperparameters

State & Action Dimensions

- `state_dim`: 42 (33 original + 3 spend + 3 audience + 3 constraint)
- `continuous_action_dim`: 2 (bid_adjustment, budget_adjustment)
- `discrete_action_dims`: [4, 4] (audience, creative)
- `hidden_dim`: 256 | `num_hidden_layers`: 3

Regularization

- `use_layer_norm`: True | `dropout_rate`: 0.1
- `activation`: "relu"

SAC Parameters

- `gamma`: 0.99 (discount factor)
- `tau`: 0.005 (soft target update)
- `alpha`: 0.2 (entropy coefficient)
- `auto_entropy_tuning`: True
- `actor_lr` / `critic_lr` / `alpha_lr`: 3e-4

Hybrid Target Entropy

Computed as:

$$-\dim(A_{\text{cont}}) + 0.98 \cdot \sum \ln(d_i)$$

where d_i are discrete action dimensions.

GuardrailConfig — Safety Bounds

Bid Guardrails

- Max increase: **+50%** | Max decrease: **-30%**
- Min bid: \$0.01 | Max bid: \$100.00
- Emergency stop: ROAS < 0.5 or CPA > 3× target

Budget Guardrails

- Max increase/decrease: **±30%**
- Min daily: \$10 | Max daily: \$100,000
- Max spend increase per action: \$1,000

Rate Limits & Confidence

- Cooldown: **4 hours** min between actions
- Max actions/day: **6**
- Min confidence to act: **0.70** | Auto-apply: **0.85**

Rollback

- Auto-rollback enabled | Observation window: 24h
- Rollback threshold: **-15%** performance
- Exploration: max 0.20, min 0.01, decay 0.995/step

RewardConfig — Multi-Objective Weights

Primary Weights



ROAS Weight



CPA Weight



Conversion Weight



CTR Weight

Penalties & Bonuses

- Budget violation: **-1.0** | CPA violation: **-0.5** | ROAS violation: **-0.5**
- Action magnitude penalty: **-0.01 × (|bid| + |budget|)**
- Action frequency penalty: **-0.05** within cooldown
- LTV bonus weight: **0.15** | Efficiency bonus: **0.10**
- ROAS target: **2.0×** | Excellent: **4.0×**

Normalization

Running normalization via Welch's online algorithm. Reward clipped to **±10**.

State, Action & Data Structures

state_action.py — 570 lines. The **CampaignState** is a 42-dimensional vector; all values normalized to approximately [0, 1].

CampaignState — 42 Dimensions

- **Core metrics [0–5]:** ctr, cvr, roas, cpa, cpc, cpm
- **Volume [6–9]:** spend_velocity, impression_volume, click_volume, conversion_volume
- **Temporal [10–15]:** hour_of_day, day_of_week, day_of_month, is_weekend, is_holiday, days_remaining
- **Trends [16–20]:** ctr/cvr/roas/cpa/spend trend_7d
- **Competitive [21–23]:** impression_share, auction_pressure, competitive_position
- **ML features [24–28]:** audience_quality, creative_fatigue, predicted_cvr, predicted_ltv, propensity_score
- **Context [29–35]:** goal encoding, platform encoding, maturity, budget_utilization, log spend dims
- **Audience/Constraints [36–41]:** segment_count, top_segment_roas, avg_frequency, target_cpa_norm, min_roas_norm, budget_limit_norm

ActionSpace — Hybrid

- **bid_adjustment:** float [-1, 1]
- **budget_adjustment:** float [-1, 1]
- **audience_action:** int [0–3] — HOLD/EXPAND/REFINE/EXCLUDE
- **creative_action:** int [0–3] — HOLD/ROTATE/PAUSE/TEST_NEW
- **confidence:** float [0, 1]
- **q_value:** float (critic estimate)

DRLDirective — DRL→LLM Bridge

Strategic output constraining LLM tactical execution. DRL decides **WHAT**; LLM decides **HOW**. Includes budget_allocation, bid_strategy, audience_priority, creative_direction, messaging_tone, urgency_level, max_offer_discount (ROAS-gated: 0.25/0.15/0.05).

Transition (Replay Record)

42-dim state, continuous [2] + discrete [2] actions, scalar reward, next_state, done flag, td_error, priority for PER.

Network Architectures

`networks.py` — 362 lines. All networks built via `create_mlp()` supporting `relu`, `tanh`, `leaky_relu`, `elu`, `gelu` activations with `LayerNorm` and `Dropout`.

ActorNetwork

Input: state [batch, 42]

Shared backbone: 42→256→256

Heads: mean [256→2], log_std [256→2, clamped -20...2], discrete logits [256→4] ×2

Output: continuous (Gaussian + tanh squash), discrete (Gumbel-Softmax), log_prob, entropy

CriticNetwork (Twin Q)

Input: concat(state, continuous, one_hot(discrete)) [batch, 52]

Architecture: 52→256→256→256→1 (×2 independent Q-networks)

Returns (q1, q2) for double-Q trick. Auto-detects discrete format.

ValueNetwork

Input: state [batch, 42]

Architecture: 42→256→256→256→1

Optional SAC v1 state value network. Returns v [batch, 1].

Factory `create_networks(config)` returns (**actor**, **critic**, **critic_target**) where `critic_target` is initialized as a copy of `critic`. Orthogonal weight init: gain=√2 for hidden, gain=0.01 for output heads.

Core Agent: SAC Implementation

`sac_agent.py` — 573 lines. Soft Actor-Critic with hybrid action spaces and Conservative Q-Learning (CQL).

01

Sample from PER Buffer

Stratified sampling returns (transitions, indices, IS weights)

02

Update Critic

TD target: $r + \gamma(1-d)(\min(Q1', Q2') - \alpha \cdot \log_prob')$. MSE loss weighted by IS weights + optional CQL loss.

03

Update Actor

Maximize $\min(Q1, Q2) - \alpha \cdot \log_prob$

04

Update Alpha

Minimize $\alpha \cdot (\log_prob + \text{target_entropy}).\text{detach}()$

05

Soft Target Update

Polyak averaging: $\theta_target = \tau \cdot \theta + (1-\tau) \cdot \theta_target$. Update PER priorities with new TD errors.

Replay Buffers & Training Config

PrioritizedReplayBuffer (PER)

- Capacity: **1,000,000** transitions
- SumTree + MinTree for $O(\log n)$ priority sampling
- Alpha: 0.6 (prioritization exponent)
- Beta: 0.4→1.0 annealed over 100,000 steps
- Stratified sampling: divides total priority into batch_size segments
- Priority update: $|td_error| + \epsilon$ raised to alpha

UniformReplayBuffer

Simple deque-based fallback with identical interface. Factory `create_replay_buffer()` selects based on `use_per` flag.

TrainingConfig Key Parameters

- `batch_size`: 256 | `replay_buffer_size`: 1M
- `min_buffer_size`: 10,000 | `num_offline_epochs`: 100
- `use_cql`: True | `cql_alpha`: 1.0
- `use_per`: True | `per_alpha`: 0.6
- `gradient_clip`: 1.0 | `weight_decay`: 1e-4
- `initial_exploration_steps`: 10,000
- `early_stopping_patience`: 10 epochs

CQL Loss

Samples `cql_num_samples` random + policy actions. Computes $\log\text{sumexp}(Q_random, Q_policy) - Q_data$ with optional importance sampling correction.

Reward Functions — 11 Components

`reward_functions.py` — 593 lines. `RewardComputer.compute(prev_state, action, curr_state, context)` returns a `MultiObjectiveReward`.



Primary Reward

Goal-specific improvement: $\text{ROAS delta} / \text{prev ROAS}$; CPA inverted (lower=better); conversion volume delta; CTR delta.



Constraint Alignment

CPA within $\pm 5\%$ of target: +0.1 bonus; 10%+ over: -0.1. ROAS within $\pm 5\%$ of min: +0.1; 10%+ under: -0.1. Budget within $\pm 2\%$: +0.05; 10%+ over: -0.15.



Efficiency & Volume Bonuses

Efficiency: $0.1 \times (\text{roas} - 2.0)$ if ROAS > target, extra at 4.0+. Volume: $0.05 \times \log_{10}(\text{conv_increase})$. LTV: $0.15 \times \Delta \text{predicted_ltv}$.



Spend Efficiency

Diminishing returns penalty when utilization > 0.7 threshold.
Efficiency-at-scale bonus when ROAS > 1.5:
 $+\text{weight} \times \text{roas} \times \log_{10}(\text{spend})$.

Training Pipelines

Offline CQL Training

`offline_trainer.py` — 654 lines. `OfflineDataExtractor` builds transitions from historical CSV/campaign data. Trends via linear regression on rolling windows. Goal encoding: $\text{roas}=0.0$, $\text{cpa}=0.25$, $\text{conversions}=0.5$, $\text{ctr}=0.75$. Platform: $\text{meta}=0.0$, $\text{google}=0.2$, $\text{tiktok}=0.4$, $\text{amazon}=0.6$, $\text{walmart}=0.8$. Epoch loop with early stopping ($\text{patience}=10$).

BigQuery Offline Training

`train_bigquery_offline.py` — 394 lines. End-to-end: BigQuery → transitions → behavioral cloning → CQL. Action inference: bid from CPC delta, budget from $0.6 \times \text{volume_change} + 0.4 \times \text{spend_velocity_change}$. Behavior cloning pretrain (5 epochs): MSE on continuous, cross-entropy on discrete. Config: $\text{state_dim}=42$, $\text{cql_alpha}=0.5$, $\text{batch_size}=256$.

Mock Training

`train.py` — 192 lines. `MockCampaignEnv`: bid increases volume but raises CPA/CTR; audience refine +5% CVR; expand -5% CVR; creative rotate +2% CTR. Reward: $(\text{revenue} - \text{cost})/100$. 50 episodes × 100 steps, $\text{batch_size}=64$.

X-Model Training (M4)

`x_training.py` — 248 lines. `XModelReplayBuffer`: list-based, capacity=100K. Behavior cloning pretrain: MSE on softmax allocation weights. SAC updates with periodic logging. Default $\text{min_buffer_size}=50$.

Safety & Production Layer

`safe_agent.py` — 610 lines. `SafeDRLAgent` wraps `SACAgent` with validation, bounded exploration, audit trail, and rollback.



SafeDRLAgent Features

- Bounded exploration: Gaussian noise clamped ± 0.2 (bid), ± 0.15 (budget), 10% discrete
- Audit trail: circular buffer of last 10,000 actions
- ActionStatus: APPROVED / MODIFIED / BLOCKED / REQUIRES_REVIEW

RollbackManager

- Observation window: **24 hours**
- Performance threshold: **-15%**
- Monitors pending observations for degradation
- Auto-rollback to prior campaign parameters

Cross-Platform Optimization

`cross_platform_optimizer.py` — 1,619 lines. Platforms: META, GOOGLE, TIKTOK, AMAZON, WALMART (encodings: 0.0, 0.2, 0.4, 0.6, 0.8).

CrossPlatformConfig

- Max single shift/cycle: **20%** | Floor: **5%** | Ceiling: **80%**
- Rebalance cooldown: **24 hours**
- Weights: ROAS 0.50, Volume 0.25, Momentum 0.15, Diversification 0.10
- Emergency ROAS floor: **0.5** (force low marginal)
- Min confidence for shift: **0.60**

7-Phase `optimize_portfolio()` Pipeline

1. Build portfolio snapshot (PlatformPerformanceTracker)
2. Check cooldown (24h)
3. Estimate marginal returns per platform
4. Emergency override: $\text{ROAS} < 0.5 \rightarrow$ force low marginal
5. Solve allocation (X-Model or heuristic BudgetAllocator)
6. Compute projected impact (ROAS from marginal rates)
7. Adjust campaign budgets, apply audience constraints, run per-campaign DRL

MarginalReturnEstimator & BudgetAllocator

Log-Linear Response Model

$$\text{Revenue}(\text{spend}) = a \cdot \ln(1 + \text{spend}/b)$$

$$\text{Marginal ROAS} = \frac{a}{b + \text{spend}}$$

Captures diminishing returns. 90-day history cap. EMA smoothing $\alpha=0.3$. Confidence = $\min(1, n/14) \times \max(0.3, 1-CV)$.

BudgetAllocator — Constrained Simplex Optimization

Projected gradient ascent maximizing:

$$\max \sum_p \left[w_{\text{roas}} \cdot \text{marginal_roas}_p \cdot b_p + w_{\text{vol}} \cdot \ln(1 + \text{conv}_p) \right]$$

Constraints: sum=total_budget; min 5% ≤ each platform ≤ 80%;

max 120% shift per week. Softmax temperature = 0.5

X-Model: Cross-Platform DRL Agent

x_model.py — 559 lines. Dedicated SAC agent for portfolio-level allocation across 5 platforms.

XModelState — 70 Dimensions

Per-platform features ($13 \times 5 = 65$): roas (/5), cpa_inv, ctr, cvr, marginal_roas, spend_share, spend_trend_7d, roas_trend_7d, conversion_trend_7d, num_campaigns_norm, segment_count_norm, top_segment_roas, avg_frequency_ratio.

Global features (5): portfolio_roas, total_budget_log, budget_utilization, portfolio_HHI (Herfindahl-Hirschman Index), active_platform_ratio.

XModelActor Architecture

Input: [batch, 70]

Layer 1: Linear(70→256) → LayerNorm → ReLU

Layer 2: Linear(256→256) → LayerNorm → ReLU

Layer 3: Linear(256→128) → LayerNorm → ReLU

mean_head: Linear(128→5) | log_std_head: Linear(128→5) [-5...2]

Output: Gaussian reparameterization → **softmax normalization** → allocation weights summing to 1.0

XModelCritic (Twin Q)

Input: concat(state, action) [batch, 75] → 256→256→1 (×2). Confidence = $1 - \text{entropy}/\text{max_entropy}$. Min/max share constraints: 5%/80%, re-normalized to sum=1.0.

Platform Model Registry (P-Models)

`platform_model_registry.py` — 308 lines. One `SACAgent` per platform with global fallback. Supports: meta, google, tiktok, amazon, walmart.



Registry Management

`get_or_create(platform)` — returns existing or creates new agent.

`load_all(checkpoint_dir)` — loads all saved platforms.

`ensure_global_fallback()` — lazy loads fallback agent.



M2 Execution

`select_action(platform, state)` — routes to platform-specific agent or global fallback.

`evaluate_q(platform, state, action)` — Q-value for X-Model value estimation.



Persistence & Diagnostics

`save/load(platform, checkpoint_dir)` — per-platform checkpoints. `get_diagnostics()` — per-platform step counts and training timestamps.

`PlatformModelMeta` tracks `state_dim=42`, `training_steps`, `last_trained_at`.

Cross-Platform DRL Engine (Orchestrator)

`cross_platform_drl_engine.py` — 832 lines. Top-level orchestrator wrapping `CrossPlatformOptimizer`, `XModelAgent`, `XModelTrainer`, and `XTrainingDataBuilder`.

DRL_PRIMARY

X-Model first. If `ModelReadinessChecker` passes (≥ 500 steps, confidence ≥ 0.40), use X-Model allocation; otherwise fall back to heuristic `BudgetAllocator`.

HEURISTIC_PRIMARY

Always use heuristic allocation. Collect X-Model training data in background for future readiness.

DUAL_BENCHMARK

Run both X-Model and heuristic. Apply DRL allocation. Log `DualRunResult` (L1 divergence, projected ROAS comparison) for benchmarking.

Automatic Training Lifecycle

Snapshot counting after each `optimize_portfolio()` call. Retrain trigger: snapshots ≥ 50 AND transitions ≥ 30 . Calls `XModelTrainer.train()` then resets counter. Integrates narrative generation and A/B test creation.

Continuous Learning Engine

`continuous_learning.py` — 740 lines. Hybrid mode runs all three learning modes simultaneously.

1

Online Mode

Single gradient step per every 4 outcomes. Low latency, continuous adaptation.

2

Batch Mode

10 gradient steps every 3,600 seconds (1 hour). Minimum batch size: 64.

3

Triggered Mode

50 gradient steps on drift detection: performance degradation -15% or mean TD error > 1.0 .

OutcomeTracker

Asynchronous outcome collection with delay profiles: impressions (minutes), clicks (hours), conversions (days). 24h pending window, 10,000 completed queue max.

ModelVersionManager

`save_version(agent, metrics)`, `load_version(version_id)`, `get_best_version(metric_name)`. Integrates ForecastFeedbackLoop for periodic forecaster refitting.

A/B Testing Framework

`ab_testing.py` — 1,114 lines. Full experiment lifecycle with statistical rigor.

ExperimentConfig Defaults

- Variants: baseline (80%), drl (20%)
- Assignment: DETERMINISTIC (SHA256 hash)
- Target sample size: 1,000
- Min detectable effect: 5% | Significance: 0.05 | Power: 0.8
- Primary metric: ROAS | Secondary: CPA, conversions, CTR

Statistical Analysis

- Welch's t-test (unequal variance)
- Welch-Satterthwaite degrees of freedom
- Confidence intervals + lift estimate
- Sample size computation for desired power

DRLABTestManager

- `create_platform_experiment()` — P-Model vs Global agent
- `create_portfolio_experiment()` — X-Model vs Heuristic
- Human-readable narratives via ParameterGlossary

Explainable AI (xAI) Narrator

`xai_narrator.py` — 668 lines. Generates plain-English explanations at both campaign and portfolio levels.

RunNarrative (Campaign)

Situation summary (trends, fatigue, utilization) → Decision summary (bid/budget/audience/creative in plain language) → Reasoning bullets ("trigger → action → because") → Confidence explanation → Reasonability check against guardrails.

PortfolioNarrative

Portfolio summary (ROAS, HHI concentration, utilization) → Allocation decision (platform shifts with amounts) → Per-platform reasoning → Confidence (Q-value mapped to language) → Risk assessment: flags HHI > 0.3, shifts > 10%, low confidence.

ParameterGlossary — 9 Key Parameters

ROAS (1.5×–5.0×), CPA (\$5–\$100), CTR (0.5%–5%), Budget Utilization (70%–95%), Creative Fatigue (0–0.3), Strategic Confidence (0.7–1.0), Allocation Weight (5%–50%), Portfolio HHI (0.2–0.5), Marginal ROAS (0.5×–3.0×). Each entry includes full name, definition, formula, normal range, and impact.

BigQuery Data Loader

`bigquery_loader.py` — 208 lines. Extracts and normalizes the 42 features matching CampaignState dimensions from Google BigQuery.

01

Connect & Query

`client()` creates BigQuery client with GCP credentials. `fetch_dataframe(where, limit)` executes SQL with optional filters. Handles (column, alias) tuples via `resolve_features()`.

02

Normalize

`fit_normalizer(df)` computes per-column mean/std. `transform(df, mean_dict, std_dict)` applies standardization with NaN fill (0.0).

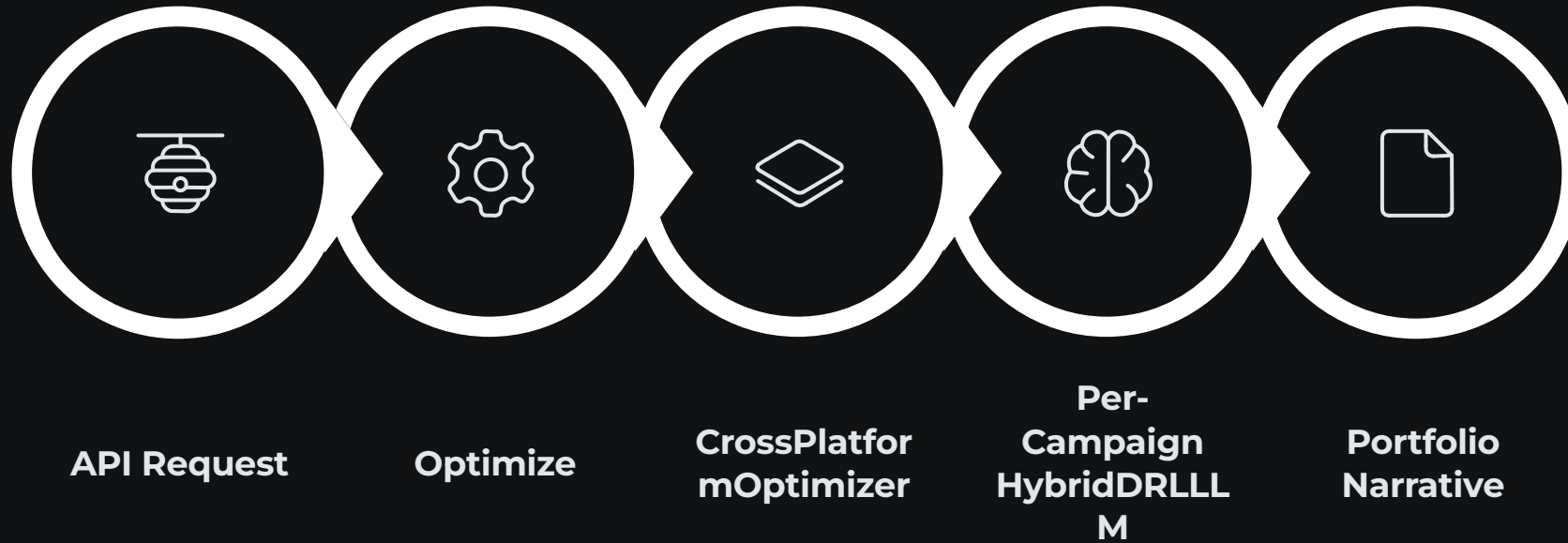
03

Load States

`load_states(where, limit)` — end-to-end pipeline: fetch + normalize → returns (ndarray [N×42], feature_list). Column aliases handle schema variations (e.g., audience_quality OR audience_quality_score).

Full Optimization Pipeline — Data Flow

End-to-end flow for a single portfolio optimization request.



Allocation Decision Logic

If X-Model available: `build_x_state(portfolio) → XModelState [70-dim] → XModelAgent.select_allocation() → AllocationRecommendation list`. Else: `BudgetAllocator.allocate() → heuristic allocation`.

M2 Per-Platform Execution

For each platform: `PlatformModelRegistry.get(platform) → SACAgent`. For each campaign: `HybridDRLLLOptimizer.optimize() → 4-phase pipeline → OptimizationResult`.

Training & Continuous Learning Data Flows

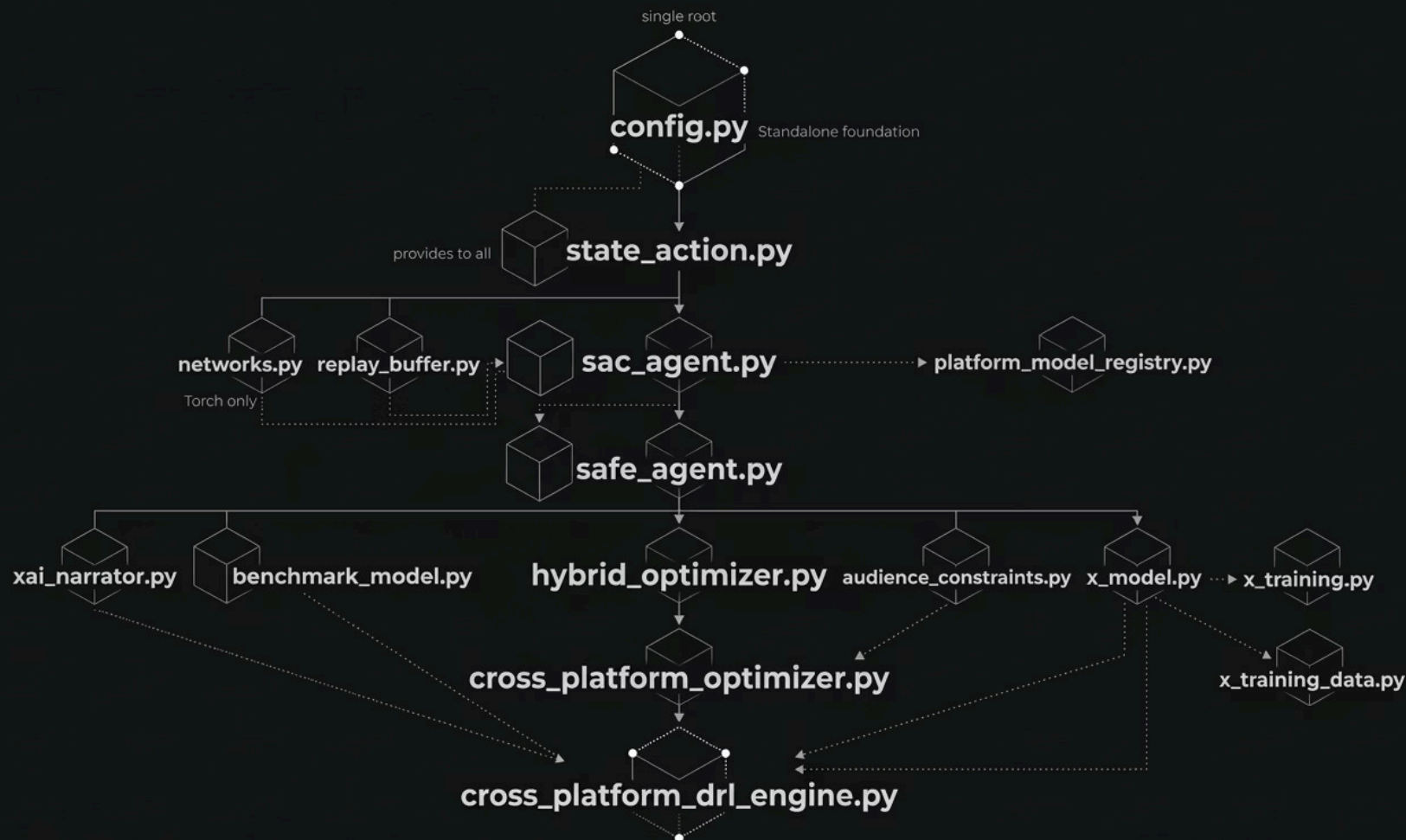
Training Data Flow

1. Historical data (BigQuery/CSV) →
`BigQueryDataLoader.load_states()` → normalized feature matrix
2. `build_transitions()` → `List[Transition]`
3. `behavior_cloning_pretrain()` — warm-start SAC
4. `OfflineTrainer.train()` — CQL offline training
5. SACAgent (per-platform) → `PlatformModelRegistry.save()`
6. Portfolio snapshots →
`XTrainingDataBuilder.record_snapshot()` → `XTransition`
7. `XModelTrainer.train()` → `XModelAgent` update

Continuous Learning Flow

1. Production action → `OutcomeTracker.record_action()`
2. [Waits for delayed outcome — minutes to days]
3. `OutcomeTracker.record_outcome()` → `Transition`
4. `ReplayBuffer.push(transition)`
5. `ContinuousLearningEngine`: Online (every 4), Batch (every 60min), Triggered (on drift)
6. `PerformanceMonitor`: -15% degradation or TD error > 1.0 → triggered update
7. `ModelVersionManager`: checkpoint + rollback |
`ForecastFeedbackLoop`: periodic refit

Key Inter-Module Dependencies



Public API — Key Exports

`__init__.py` — 150 lines, exports 60+ symbols across all modules.

Config

DRLConfig, TrainingConfig,
GuardrailConfig,
AllocationStrategy,
CrossPlatformStrategyConfig

State/Action

CampaignState,
ActionSpace, DRLDirective,
Transition

Agent & Safety

SACAgent, SafeDRLAgent,
ActionValidator,
PrioritizedReplayBuffer,
CQLLoss, RewardComputer

Cross-Platform

CrossPlatformOptimizer,
CrossPlatformResult,
PlatformPortfolio,
MarginalReturnEstimator,
BudgetAllocator, Platform

X & P Models

XModelAgent, XModelState, XModelAction, build_x_state,
XModelTrainer, XTrainingDataBuilder, PlatformModelRegistry,
CrossPlatformDRL engine, DualRunResult

Learning & xAI

ContinuousLearningEngine, DRLABTestManager,
OptimizationNarrator, PortfolioNarrative,
AudienceConstraintManager, CampaignForecaster,
ForecastFeedbackLoop